2(a)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        printf("Child : PID = %d, Parent PID = %d\n", getpid(), getppid());
        exit(0);
    }
    wait(NULL);
    printf("Parent: PID = %d, Child PID  = %d\n", getpid(), pid);
    return 0;
}
```

2(b)

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Current date and time:\n");
    fflush(stdout);                 /* exec replaces the buffer */
    execlp("date", "date", NULL);
    perror("execlp");               /* runs only if exec fails */
    return 1;
}
```

2(c)

```
#include <stdio.h>
#include <time.h>
#include <sys/stat.h>

int main() {
    char name[100];
    struct stat s;

    printf("Enter file name: ");
    if (scanf("%99s", name) != 1) return 1;
    if (stat(name, &s) == -1) { perror("stat"); return 1; }

    printf("Size        : %lld bytes\n", (long long)s.st_size);
    printf("Inode       : %llu\n", (unsigned long long)s.st_ino);
    printf("Mode        : %o\n", (unsigned)s.st_mode);
    printf("Links       : %lu\n", (unsigned long)s.st_nlink);
    printf("UID / GID   : %d / %d\n", (int)s.st_uid, (int)s.st_gid);
    printf("Last access : %s", ctime(&s.st_atime));
    printf("Last modify : %s", ctime(&s.st_mtime));
    return 0;
}
```

2(d)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    printf("Before fork\n");
    fflush(stdout);                 /* avoid printing it twice */

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        printf("\nChild executing...\n");
        printf("Child PID  : %d\nParent PID : %d\n", getpid(), getppid());
        exit(0);
    }
    wait(NULL);                     /* parent waits for child */
    printf("\nParent resumed...\n");
    printf("Parent PID : %d\nChild PID  : %d\n", getpid(), pid);
    return 0;
}
```

2(e)

```
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    char msg[] = "Operating Systems Lab", buf[50];

    int fd = creat("sample.txt", 0644);
    if (fd < 0) { perror("creat"); return 1; }
    write(fd, msg, strlen(msg));
    close(fd);

    fd = open("sample.txt", O_RDONLY);
    if (fd < 0) { perror("open"); return 1; }
    int n = read(fd, buf, sizeof buf - 1);
    close(fd);

    buf[n > 0 ? n : 0] = '\0';
    printf("File contents: %s\n", buf);
    return 0;
}
```

3(a)

```
#include <stdio.h>

int main() {
    int n, pid[20], bt[20], wt = 0, tat, twt = 0, ttat = 0;

    printf("Number of processes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("PID and burst time of P%d: ", i + 1);
        scanf("%d %d", &pid[i], &bt[i]);
    }

    printf("\nPID\tBT\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        tat = wt + bt[i];
        printf("%d\t%d\t%d\t%d\n", pid[i], bt[i], wt, tat);
        twt += wt;
        ttat += tat;
        wt = tat;                    /* next one waits till now */
    }
    printf("Average WT  = %.2f\n", (float)twt / n);
    printf("Average TAT = %.2f\n", (float)ttat / n);
    return 0;
}
```

3(b)

```
#include <stdio.h>

int main() {
    int n, pid[20], bt[20], t, wt = 0, tat, twt = 0, ttat = 0;

    printf("Number of processes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("PID and burst time of P%d: ", i + 1);
        scanf("%d %d", &pid[i], &bt[i]);
    }

    for (int i = 0; i < n - 1; i++)     /* sort by burst time */
        for (int j = i + 1; j < n; j++)
            if (bt[j] < bt[i]) {
                t = bt[i];  bt[i] = bt[j];  bt[j] = t;
                t = pid[i]; pid[i] = pid[j]; pid[j] = t;
            }

    printf("\nPID\tBT\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        tat = wt + bt[i];
        printf("%d\t%d\t%d\t%d\n", pid[i], bt[i], wt, tat);
        twt += wt;
        ttat += tat;
        wt = tat;
    }
    printf("Average WT  = %.2f\n", (float)twt / n);
    printf("Average TAT = %.2f\n", (float)ttat / n);
    return 0;
}
```

3(c)

```
#include <stdio.h>

int main() {
    int n, q, pid[20], bt[20], rt[20], time = 0, done = 0;
    float twt = 0, ttat = 0;

    printf("Number of processes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("PID and burst time of P%d: ", i + 1);
        scanf("%d %d", &pid[i], &bt[i]);
        rt[i] = bt[i];
    }
    printf("Time quantum: ");
    scanf("%d", &q);

    printf("\nPID\tBT\tWT\tTAT\n");
    while (done < n)
        for (int i = 0; i < n; i++) {
            if (rt[i] == 0) continue;
            int run = rt[i] < q ? rt[i] : q;
            time += run;
            rt[i] -= run;
            if (rt[i] == 0) {           /* process finished */
                done++;
                printf("%d\t%d\t%d\t%d\n",
                       pid[i], bt[i], time - bt[i], time);
                twt += time - bt[i];
                ttat += time;
            }
        }
    printf("Average WT  = %.2f\n", twt / n);
    printf("Average TAT = %.2f\n", ttat / n);
    return 0;
}
```

3(d)

```
#include <stdio.h>

struct proc { int pid, bt, pri; };

int main() {
    struct proc p[20], t;
    int n, wt = 0, tat, twt = 0, ttat = 0;

    printf("Number of processes: ");
    scanf("%d", &n);
    printf("(lower number = higher priority)\n");
    for (int i = 0; i < n; i++) {
        printf("PID, burst time, priority of P%d: ", i + 1);
        scanf("%d %d %d", &p[i].pid, &p[i].bt, &p[i].pri);
    }

    for (int i = 0; i < n - 1; i++)     /* sort by priority */
        for (int j = i + 1; j < n; j++)
            if (p[j].pri < p[i].pri) { t = p[i]; p[i] = p[j]; p[j] = t; }

    printf("\nPID\tBT\tPri\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        tat = wt + p[i].bt;
        printf("%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].bt, p[i].pri, wt, tat);
        twt += wt;
        ttat += tat;
        wt = tat;
    }
    printf("Average WT  = %.2f\n", (float)twt / n);
    printf("Average TAT = %.2f\n", (float)ttat / n);
    return 0;
}
```

4(a)

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>

int main() {
    key_t key = ftok("/tmp", 65);       /* same key in reader */
    int id = shmget(key, 1024, 0666 | IPC_CREAT);
    if (id < 0) { perror("shmget"); return 1; }

    char *str = shmat(id, NULL, 0);
    if (str == (char *)-1) { perror("shmat"); return 1; }

    printf("Write data: ");
    fgets(str, 1024, stdin);
    printf("Data written to memory: %s", str);

    shmdt(str);
    return 0;
}
```

4(b)

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>

int main() {
    key_t key = ftok("/tmp", 65);       /* same key as writer */
    int id = shmget(key, 1024, 0666 | IPC_CREAT);
    if (id < 0) { perror("shmget"); return 1; }

    char *str = shmat(id, NULL, 0);
    if (str == (char *)-1) { perror("shmat"); return 1; }

    printf("Data read from memory: %s", str);

    shmdt(str);
    shmctl(id, IPC_RMID, NULL);         /* destroy segment */
    return 0;
}
```

5

```
#include <stdio.h>
#define P 5                             /* processes */
#define R 3                             /* resource types */

int main() {
    int alloc[P][R] = {{0,1,0}, {2,0,0}, {3,0,2}, {2,1,1}, {0,0,2}};
    int max[P][R]   = {{7,5,3}, {3,2,2}, {9,0,2}, {2,2,2}, {4,3,3}};
    int avail[R] = {3, 3, 2}, need[P][R], done[P] = {0}, seq[P], cnt = 0;

    printf("Need matrix\n");
    for (int i = 0; i < P; i++) {
        printf("P%d: ", i);
        for (int j = 0; j < R; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }

    for (int pass = 0; pass < P; pass++)
        for (int i = 0; i < P; i++) {
            if (done[i]) continue;
            int j = 0;
            while (j < R && need[i][j] <= avail[j]) j++;
            if (j < R) continue;        /* cannot satisfy now */
            for (j = 0; j < R; j++) avail[j] += alloc[i][j];
            done[i] = 1;
            seq[cnt++] = i;
        }

    if (cnt < P) {
        printf("\nSystem is NOT in a safe state\n");
        return 0;
    }
    printf("\nSafe sequence: ");
    for (int i = 0; i < P; i++)
        printf("P%d%s", seq[i], i < P - 1 ? " -> " : "\n");
    return 0;
}
```

6(a)

```
#include <stdio.h>

int main() {
    int f, n, pg[50], fr[10], last[10], faults = 0;

    printf("Number of frames: ");
    scanf("%d", &f);
    printf("Number of pages: ");
    scanf("%d", &n);
    printf("Reference string: ");
    for (int i = 0; i < n; i++) scanf("%d", &pg[i]);
    for (int j = 0; j < f; j++) fr[j] = last[j] = -1;

    printf("\nPage\tFrames\tStatus\n");
    for (int i = 0; i < n; i++) {
        int pos = -1;
        for (int j = 0; j < f; j++)
            if (fr[j] == pg[i]) pos = j;
        int hit = pos != -1;

        if (!hit) {                     /* empty or least recently used */
            faults++;
            pos = 0;
            for (int j = 1; j < f; j++)
                if (last[j] < last[pos]) pos = j;
            fr[pos] = pg[i];
        }
        last[pos] = i;

        printf("%d\t", pg[i]);
        for (int j = 0; j < f; j++)
            if (fr[j] < 0) printf("- "); else printf("%d ", fr[j]);
        printf("\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nPage faults = %d\nPage hits   = %d\n", faults, n - faults);
    return 0;
}
```

6(b)

```
#include <stdio.h>

int main() {
    int f, n, pg[50], fr[10], faults = 0;

    printf("Number of frames: ");
    scanf("%d", &f);
    printf("Number of pages: ");
    scanf("%d", &n);
    printf("Reference string: ");
    for (int i = 0; i < n; i++) scanf("%d", &pg[i]);
    for (int j = 0; j < f; j++) fr[j] = -1;

    printf("\nPage\tFrames\tStatus\n");
    for (int i = 0; i < n; i++) {
        int hit = 0;
        for (int j = 0; j < f; j++)
            if (fr[j] == pg[i]) hit = 1;

        if (!hit) {         /* empty frame, else one used farthest ahead */
            int pos = 0, far = -1;
            faults++;
            for (int j = 0; j < f; j++) {
                if (fr[j] < 0) { pos = j; break; }
                int k = i + 1;
                while (k < n && pg[k] != fr[j]) k++;
                if (k > far) { far = k; pos = j; }
            }
            fr[pos] = pg[i];
        }

        printf("%d\t", pg[i]);
        for (int j = 0; j < f; j++)
            if (fr[j] < 0) printf("- "); else printf("%d ", fr[j]);
        printf("\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nPage faults = %d\nPage hits   = %d\n", faults, n - faults);
    return 0;
}
```

6(c)

```
#include <stdio.h>

int main() {
    int f, n, pg[50], fr[10], next = 0, faults = 0;

    printf("Number of frames: ");
    scanf("%d", &f);
    printf("Number of pages: ");
    scanf("%d", &n);
    printf("Reference string: ");
    for (int i = 0; i < n; i++) scanf("%d", &pg[i]);
    for (int j = 0; j < f; j++) fr[j] = -1;

    printf("\nPage\tFrames\tStatus\n");
    for (int i = 0; i < n; i++) {
        int hit = 0;
        for (int j = 0; j < f; j++)
            if (fr[j] == pg[i]) hit = 1;

        if (!hit) {                     /* replace oldest page */
            fr[next] = pg[i];
            next = (next + 1) % f;
            faults++;
        }

        printf("%d\t", pg[i]);
        for (int j = 0; j < f; j++)
            if (fr[j] < 0) printf("- "); else printf("%d ", fr[j]);
        printf("\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nPage faults = %d\nPage hits   = %d\n", faults, n - faults);
    return 0;
}
```

7(a)

```
#include <stdio.h>

int main() {
    int nb, np, blk[50], prc[50], used[50] = {0};

    printf("Number of blocks: ");
    scanf("%d", &nb);
    printf("Block sizes: ");
    for (int i = 0; i < nb; i++) scanf("%d", &blk[i]);
    printf("Number of processes: ");
    scanf("%d", &np);
    printf("Process sizes: ");
    for (int i = 0; i < np; i++) scanf("%d", &prc[i]);

    printf("\nProcess\tSize\tBlock\tFragment\n");
    for (int i = 0; i < np; i++) {
        int b = -1;                     /* smallest block that fits */
        for (int j = 0; j < nb; j++)
            if (!used[j] && blk[j] >= prc[i] &&
                (b < 0 || blk[j] < blk[b])) b = j;

        if (b < 0) {
            printf("P%d\t%d\tNot allocated\n", i + 1, prc[i]);
            continue;
        }
        used[b] = 1;
        printf("P%d\t%d\tB%d\t%d\n", i + 1, prc[i], b + 1, blk[b] - prc[i]);
    }
    return 0;
}
```

7(b)

```
#include <stdio.h>

int main() {
    int nb, np, blk[50], prc[50], used[50] = {0};

    printf("Number of blocks: ");
    scanf("%d", &nb);
    printf("Block sizes: ");
    for (int i = 0; i < nb; i++) scanf("%d", &blk[i]);
    printf("Number of processes: ");
    scanf("%d", &np);
    printf("Process sizes: ");
    for (int i = 0; i < np; i++) scanf("%d", &prc[i]);

    printf("\nProcess\tSize\tBlock\tFragment\n");
    for (int i = 0; i < np; i++) {
        int b = -1;                     /* first block that fits */
        for (int j = 0; j < nb; j++)
            if (!used[j] && blk[j] >= prc[i]) { b = j; break; }

        if (b < 0) {
            printf("P%d\t%d\tNot allocated\n", i + 1, prc[i]);
            continue;
        }
        used[b] = 1;
        printf("P%d\t%d\tB%d\t%d\n", i + 1, prc[i], b + 1, blk[b] - prc[i]);
    }
    return 0;
}
```

7(c)

```
#include <stdio.h>

int main() {
    int nb, np, blk[50], prc[50], used[50] = {0};

    printf("Number of blocks: ");
    scanf("%d", &nb);
    printf("Block sizes: ");
    for (int i = 0; i < nb; i++) scanf("%d", &blk[i]);
    printf("Number of processes: ");
    scanf("%d", &np);
    printf("Process sizes: ");
    for (int i = 0; i < np; i++) scanf("%d", &prc[i]);

    printf("\nProcess\tSize\tBlock\tFragment\n");
    for (int i = 0; i < np; i++) {
        int b = -1;                     /* largest block that fits */
        for (int j = 0; j < nb; j++)
            if (!used[j] && blk[j] >= prc[i] &&
                (b < 0 || blk[j] > blk[b])) b = j;

        if (b < 0) {
            printf("P%d\t%d\tNot allocated\n", i + 1, prc[i]);
            continue;
        }
        used[b] = 1;
        printf("P%d\t%d\tB%d\t%d\n", i + 1, prc[i], b + 1, blk[b] - prc[i]);
    }
    return 0;
}
```

8(a)

```
#include <stdio.h>
#include <stdlib.h>

int cur, total;

void go(int to) {                       /* move head, add seek */
    total += abs(to - cur);
    cur = to;
    printf(" -> %d", to);
}

int main() {
    int n, r[100];

    printf("Number of requests: ");
    scanf("%d", &n);
    printf("Request queue: ");
    for (int i = 0; i < n; i++) scanf("%d", &r[i]);
    printf("Initial head position: ");
    scanf("%d", &cur);

    printf("\nSeek sequence: %d", cur);
    for (int i = 0; i < n; i++) go(r[i]);
    printf("\nTotal seek time = %d cylinders\n", total);
    return 0;
}
```

8(b)

```
#include <stdio.h>
#include <stdlib.h>

int cur, total;

void go(int to) {                       /* move head, add seek */
    total += abs(to - cur);
    cur = to;
    printf(" -> %d", to);
}

int main() {
    int n, r[100], done[100] = {0};

    printf("Number of requests: ");
    scanf("%d", &n);
    printf("Request queue: ");
    for (int i = 0; i < n; i++) scanf("%d", &r[i]);
    printf("Initial head position: ");
    scanf("%d", &cur);

    printf("\nSeek sequence: %d", cur);
    for (int c = 0; c < n; c++) {
        int k = -1;                     /* nearest pending request */
        for (int i = 0; i < n; i++)
            if (!done[i] && (k < 0 || abs(r[i] - cur) < abs(r[k] - cur)))
                k = i;
        done[k] = 1;
        go(r[k]);
    }
    printf("\nTotal seek time = %d cylinders\n", total);
    return 0;
}
```

8(c)

```
#include <stdio.h>
#include <stdlib.h>

int cur, total;

void go(int to) {                       /* move head, add seek */
    total += abs(to - cur);
    cur = to;
    printf(" -> %d", to);
}

int cmp(const void *a, const void *b) { return *(int *)a - *(int *)b; }

int main() {
    int size, n, r[100], dir, k = 0;

    printf("Number of cylinders: ");
    scanf("%d", &size);
    printf("Number of requests: ");
    scanf("%d", &n);
    printf("Request queue: ");
    for (int i = 0; i < n; i++) scanf("%d", &r[i]);
    printf("Initial head position: ");
    scanf("%d", &cur);
    printf("Direction (1 = high, 0 = low): ");
    scanf("%d", &dir);

    qsort(r, n, sizeof(int), cmp);
    while (k < n && r[k] < cur) k++;    /* r[k..] are above head */

    printf("\nSeek sequence: %d", cur);
    if (dir) {
        for (int i = k; i < n; i++) go(r[i]);
        if (cur != size - 1) go(size - 1);
        for (int i = k - 1; i >= 0; i--) go(r[i]);
    } else {
        for (int i = k - 1; i >= 0; i--) go(r[i]);
        if (cur != 0) go(0);
        for (int i = k; i < n; i++) go(r[i]);
    }
    printf("\nTotal seek time = %d cylinders\n", total);
    return 0;
}
```

9

```
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>

/* DAC: owner decides permissions; show rwx bits */
void dac(const char *file) {
    struct stat st;
    const char *who[] = {"Owner", "Group", "Others"};

    if (stat(file, &st) != 0) { printf("File not found\n"); return; }
    printf("[DAC] %s\n", file);
    for (int i = 0; i < 3; i++) {
        int m = st.st_mode >> (6 - 3 * i);
        printf("%-6s: %c%c%c\n", who[i], m & 4 ? 'r' : '-',
               m & 2 ? 'w' : '-', m & 1 ? 'x' : '-');
    }
}

/* RBAC: permissions come from the user's role */
struct role { char *name; int r, w, x; };

void rbac(const char *name, const char *op) {
    struct role roles[] = {
        {"admin", 1, 1, 1}, {"staff", 1, 1, 0}, {"guest", 1, 0, 0}
    };
    printf("\n[RBAC] Role: %s, Operation: %s\n", name, op);
    for (int i = 0; i < 3; i++) {
        if (strcmp(roles[i].name, name) != 0) continue;
        int ok = !strcmp(op, "read")  ? roles[i].r :
                 !strcmp(op, "write") ? roles[i].w : roles[i].x;
        printf("Access %s\n", ok ? "GRANTED" : "DENIED");
        return;
    }
    printf("Unknown role\n");
}

/* MAC (Bell-LaPadula): 1 public, 2 confidential, 3 secret */
void mac(int user, int file, const char *op) {
    int ok = !strcmp(op, "read") ? user >= file   /* no read up */
                                 : user <= file;  /* no write down */
    printf("\n[MAC] User %d, File %d, Operation: %s\n", user, file, op);
    printf("Access %s\n", ok ? "GRANTED" : "DENIED");
}

int main() {
    FILE *fp = fopen("testfile.txt", "w");
    if (fp) { fprintf(fp, "sample data\n"); fclose(fp); }
    chmod("testfile.txt", 0754);        /* rwx r-x r-- */
    dac("testfile.txt");

    rbac("admin", "write");
    rbac("guest", "write");
```

```
    rbac("staff", "execute");

    mac(3, 2, "read");                  /* granted */
    mac(1, 2, "read");                  /* denied  */
    mac(3, 1, "write");                 /* denied  */
    return 0;
}
```

10

```
#include <stdio.h>

int main() {
    FILE *fp;
    char line[100], name[30];
    int roll, ch, nums[5] = {10, 20, 30, 40, 50}, back[5];

    /* write text file */
    if (!(fp = fopen("student.txt", "w"))) { perror("fopen"); return 1; }
    fputs("Student Record File\n", fp);
    fprintf(fp, "%d %s\n", 101, "Arun");
    fputs("Grade: ", fp);
    fputc('A', fp);
    fputc('\n', fp);
    fclose(fp);
    printf("Text file written (fputs, fprintf, fputc)\n\n");

    /* read text file */
    if (!(fp = fopen("student.txt", "r"))) { perror("fopen"); return 1; }
    fgets(line, sizeof line, fp);
    printf("Header (fgets) : %s", line);
    fscanf(fp, "%d %29s", &roll, name);
    printf("Roll (fscanf)  : %d\nName (fscanf)  : %s\n", roll, name);
    fgetc(fp);                          /* skip newline */
    printf("Rest (fgetc)   : ");
    while ((ch = fgetc(fp)) != EOF) putchar(ch);
    fclose(fp);

    /* write and read binary file */
    if (!(fp = fopen("numbers.bin", "wb"))) { perror("fopen"); return 1; }
    fwrite(nums, sizeof(int), 5, fp);
    fclose(fp);

    if (!(fp = fopen("numbers.bin", "rb"))) { perror("fopen"); return 1; }
    fread(back, sizeof(int), 5, fp);
    fclose(fp);

    printf("\nBinary read (fread): ");
    for (int i = 0; i < 5; i++) printf("%d ", back[i]);
    printf("\n");
    return 0;
}
```